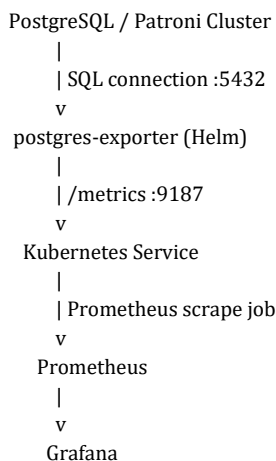


PostgreSQL + Patroni Monitoring with Prometheus and Grafana

Helm postgres-exporter + Prometheus values.yaml scrape job + Grafana

Item	Lab configuration
Working directory	E:\Postgres\patronideployment
PostgreSQL	PostgreSQL 18 / Patroni HA
Prometheus namespace	prometheus
Existing Prometheus release	prometheus
PostgreSQL exporter Helm release	postgres-exporter
Exporter chart	prometheus-community/prometheus-postgres-exporter
Exporter port	9187
Patroni metrics port	8008
Grafana	Prometheus data visualization

1. Revised Monitoring Architecture



Patroni pods :8008/metrics -----> Prometheus

This edition follows the requested Helm workflow: install postgres-exporter with datasource settings, save the exporter's chart values to a file, update/upgrade it from values.yaml, add a PostgreSQL exporter scrape job to Prometheus's Helm values, upgrade the existing Prometheus release, and visualize the collected metrics in Grafana.

2. Important Point About the Two values.yaml Files

Use separate folders/files because the two Helm charts have different schemas.

```

E:\Postgres\patronideployment\
|
+-- postgres-exporter\
|   +-- values.yaml      # postgres-exporter chart values
|
+-- prometheus\
    +-- values.yaml      # Prometheus chart values

```

Note: Do not accidentally pass the `postgres-exporter values.yaml` to the Prometheus chart, or the Prometheus `values.yaml` to the exporter chart.

3. Lab 1 - Verify Existing Components

```
kubectl get pods -n prometheus
kubectl get svc -n prometheus
kubectl get pods -n postgres-ha
kubectl get svc -n postgres-ha
helm list -A
```

Expected result: The existing prometheus-server is Running and the PostgreSQL/Patroni cluster is healthy.

4. Lab 2 - Add/Update the Prometheus Community Helm Repository

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
```

Verify the exporter chart:

```
helm search repo prometheus-community/prometheus-postgres-exporter
```

5. Lab 3 - Identify the PostgreSQL Service

Before using `postgreservice` in the exporter command, verify the actual Kubernetes Service that points to the writable PostgreSQL primary:

```
kubectl get svc -A
kubectl get svc -n postgres-ha
```

If your Service is literally `postgreservice` and the exporter is installed in the same namespace, `host=postgreservice` is sufficient. If the exporter is in another namespace, use Kubernetes DNS:

```
postgreservice.postgres-ha.svc.cluster.local
```

Note: For the Zalando operator lab, the writable Service may instead be named `acid-postgres-ha`. Substitute the actual service name.

6. Lab 4 - Install postgres-exporter Using Helm

Create and enter a separate exporter working folder:

```
cd /d E:\Postgres\patronideployment
mkdir postgres-exporter
cd postgres-exporter
```

Example matching the requested customerdb workflow:

```
helm install postgres-exporter ^
prometheus-community/prometheus-postgres-exporter ^
--set config.datasource.host=postgreservice.postgres-ha.svc.cluster.local ^
--set config.datasource.user=postgres ^
--set config.datasource.password=postgres ^
--set-string config.datasource.port="5432" ^
--set config.datasource.database=customerdb ^
--set config.datasource.sslmode=disable ^
-n postgres-ha
```

Note: Use your real PostgreSQL service, database, user and password. `sslmode=disable` is appropriate only when that matches the lab database configuration. Production credentials should come from a Kubernetes Secret rather than command history.

Verify:

```
helm list -n postgres-ha
kubectl get pods -n postgres-ha | findstr exporter
kubectl get svc -n postgres-ha | findstr exporter
```

7. Lab 5 - Generate the Exporter values.yaml

```
kubectl get secrets -n postgres-ha
```

```
kubectl get secret postgres.acid-postgres-ha.credentials.postgresql.acid.zalan.do -n postgres-ha -o yaml
```

```
find secret/password
```

password:

```
Z3F4dHY5RmdDbUh4NjRPa0RHdjIQQlVOTmdUUmZRSdRubzgyVVJiaFdTbjJad1gzVkdwdFFMTGILT2RFa0NCUQ==
```

```
username: cG9zdGdyZXM=
```

decode password using bash shell

```
kubectl get secret postgres.acid-postgres-ha.credentials.postgresql.acid.zalan.do \
```

```
-n postgres-ha \
```

```
-o jsonpath='{.data.password}' | base64 -d
```

```
gqxtv9FgCmHx64OkDGv9PBUNNgTRfQH4no82URbhWSn2ZwX3VGptQLLiKOEKCBQ
```

From the postgres-exporter folder:

```
helm show values prometheus-community/prometheus-postgres-exporter > values.yaml
notepad values.yaml
```

Find the datasource section and configure it for the lab:

config:

```
## The datasource properties on config are passed through helm tpl function.
```

```
## ref: https://helm.sh/docs/developing_charts/#using-the-tpl-function
```

datasource:

```
# Specify one of both datasource or datasourceSecret
```

```
host: 'acid-postgres-ha'
```

```
user: postgres
```

```
# Specify userFile if DB username is stored in a file.
```

```
# For example, to use with vault-injector from Hashicorp
```

```
userFile: "
```

```
userSecret: {}
```

```
# Secret name
```

```
# name:
```

```
# User key inside secret

# key:

# Only one of password, passwordFile, passwordSecret and pgpassfile can be specified
password: ""

# Specify passwordFile if DB password is stored in a file.

# For example, to use with vault-injector from Hashicorp
passwordFile: ""

# Specify passwordSecret if DB password is stored in secret.

passwordSecret:

  name: postgres.acid-postgres-ha.credentials.postgresql.acid.zalan.do

  key: password

# Secret name

# name:

# Password key inside secret

# key:

pgpassfile: ""

# If pgpassfile is set, it is used to initialize the PGPASSFILE environment variable.

# See https://www.postgresql.org/docs/14/libpq-pgpass.html for more info.

port: "5432"

database: 'postgres'

sslmode: require
```

Upgrade the exporter using this file:

```
helm upgrade postgres-exporter prometheus-community/prometheus-postgres-exporter -n postgres-ha -f values.yaml
```

Expected result: The exporter pod restarts/updates if required and becomes Running.

```

pg_statio_user_tables_idx_blocks_read{datname="postgres",relname="postgres_log",schema
# HELP pg_statio_user_tables_tidx_blocks_hit Number of buffer hits in this table's TOA
# TYPE pg_statio_user_tables_tidx_blocks_hit counter
pg_statio_user_tables_tidx_blocks_hit{datname="postgres",relname="job",schemaname="cron"} 0
pg_statio_user_tables_tidx_blocks_hit{datname="postgres",relname="job_run_details",schemaname="cron"} 0
pg_statio_user_tables_tidx_blocks_hit{datname="postgres",relname="postgres_log",schemaname="public"} 0
# HELP pg_statio_user_tables_tidx_blocks_read Number of disk blocks read from this table's TOAST table indexes (if any)
# TYPE pg_statio_user_tables_tidx_blocks_read counter
pg_statio_user_tables_tidx_blocks_read{datname="postgres",relname="job",schemaname="cron"} 0
pg_statio_user_tables_tidx_blocks_read{datname="postgres",relname="job_run_details",schemaname="cron"} 0
pg_statio_user_tables_tidx_blocks_read{datname="postgres",relname="postgres_log",schemaname="public"} 0
# HELP pg_statio_user_tables_toast_blocks_hit Number of buffer hits in this table's TOAST table (if any)
# TYPE pg_statio_user_tables_toast_blocks_hit counter
pg_statio_user_tables_toast_blocks_hit{datname="postgres",relname="job",schemaname="cron"} 0
pg_statio_user_tables_toast_blocks_hit{datname="postgres",relname="job_run_details",schemaname="cron"} 0
pg_statio_user_tables_toast_blocks_hit{datname="postgres",relname="postgres_log",schemaname="public"} 0
# HELP pg_statio_user_tables_toast_blocks_read Number of disk blocks read from this table's TOAST table (if any)
# TYPE pg_statio_user_tables_toast_blocks_read counter
pg_statio_user_tables_toast_blocks_read{datname="postgres",relname="job",schemaname="cron"} 0
pg_statio_user_tables_toast_blocks_read{datname="postgres",relname="job_run_details",schemaname="cron"} 0
pg_statio_user_tables_toast_blocks_read{datname="postgres",relname="postgres_log",schemaname="public"} 0
# HELP pg_up Whether the last scrape of metrics from PostgreSQL was able to connect to the server (1 for yes, 0 for no).
# TYPE pg_up gauge
pg_up 1
# HELP pg_wal_segments Number of WAL segments
# TYPE pg_wal_segments gauge
pg_wal_segments 5
# HELP pg_wal_size_bytes Total size of WAL segments
# TYPE pg_wal_size_bytes gauge
pg_wal_size_bytes 8.388608e+07
# HELP postgres_exporter_build_info A metric with a constant '1' value labeled by version, revision, branch, goversion from which postgres_exporter was built, and
and goarch for the build.
# TYPE postgres_exporter_build_info gauge
postgres_exporter_build_info{branch="HEAD",goarch="amd64",goos="linux",goversion="go1.26.4",revision="867fbcac31cd18c143e244190ea9168cca069827",tags="unknown",ver
"} 1
# HELP postgres_exporter_config_last_reload_success_timestamp_seconds Timestamp of the last successful configuration reload.
# TYPE postgres_exporter_config_last_reload_success_timestamp_seconds gauge
postgres_exporter_config_last_reload_success_timestamp_seconds 0
# HELP postgres_exporter_config_last_reload_successful Postgres exporter config loaded successfully.
# TYPE postgres_exporter_config_last_reload_successful gauge
postgres_exporter_config_last_reload_successful 0
# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds

```

8. Lab 6 - Test the Exporter

First use the Service created by the Helm chart:

```
kubectl get svc -n postgres-ha | findstr postgres-exporter
```

Port-forward is the simplest localhost test:

```
kubectl port-forward -n postgres-ha svc/postgres-exporter-prometheus-postgres-exporter 9187:80
```

Note: Confirm the Service port using `kubectl get svc` before port-forwarding; chart versions can differ.

In another terminal:

```
curl http://localhost:9187/metrics
```

Expected result: Exporter metrics are returned; `pg_up` should normally be 1 when the database connection succeeds.

9. Optional - Expose Exporter with LoadBalancer

The requested command can expose the exporter Deployment externally:

```

kubectl expose deployment ^
-n postgres-ha ^
--port 9187 ^
--target-port 9187 ^
postgres-exporter-prometheus-postgres-exporter ^
--type=LoadBalancer ^
--name=postgresexporterservice

```

Verify:

```
kubectl get svc -n postgres-ha postgresexporterservice
```

Note: A LoadBalancer is not required for Prometheus when both Prometheus and the exporter run in the same Kubernetes cluster. Prometheus can scrape the exporter's ClusterIP/Kubernetes DNS directly. On Docker Desktop, `host.docker.internal` can work for host-reachable services, but in-cluster DNS is cleaner and more portable.

10. Lab 7 - Create a Separate Prometheus values.yaml Folder

```
cd /d E:\Postgres\patronideployment
mkdir prometheus
cd prometheus
```

Preserve the values currently applied to the running Prometheus release:

```
helm get values prometheus -n prometheus -o yaml > values.yaml
notepad values.yaml
```

Note: *This is safer than starting with a fresh chart-default values file because Prometheus is already deployed.*

11. Lab 8 - Add postgres_exporter Job to Prometheus values.yaml

Yes—the scrape job can be managed through the Prometheus Helm values file. For current prometheus-community/prometheus chart versions, extraScrapeConfigs is a supported way to append jobs without replacing the default Kubernetes scrape jobs.

Recommended in-cluster configuration:

```
extraScrapeConfigs: |
- job_name: 'postgres_exporter'
  static_configs:
  - targets:
    - 'postgres-exporter-prometheus-postgres-exporter.postgres-ha.svc.cluster.local:80'
```

Note: *Use kubectl get svc -n postgres-ha to confirm the exporter's Service port. If it exposes 9187 instead of 80, use :9187.*

If you specifically expose the exporter to the Docker Desktop host and want to follow the host.docker.internal pattern, the requested job is:

```
extraScrapeConfigs: |
- job_name: 'postgres_exporter'
  static_configs:
  - targets:
    - 'host.docker.internal:9187'
```

The first version is preferred for Kubernetes-to-Kubernetes communication.

12. Alternative for Newer Prometheus Chart Values

Recent prometheus-community/prometheus chart releases also support adding a native scrapeConfigs entry:

```
scrapeConfigs:
  postgres_exporter:
    static_configs:
    - targets:
      - postgres-exporter-prometheus-postgres-exporter.postgres-ha.svc.cluster.local:80
```

Note: *Do not replace the existing scrapeConfigs map with only this job if your current values already customize the defaults. extraScrapeConfigs is convenient when you simply want to append the exporter target.*

13. Lab 9 - Upgrade the Existing Prometheus Release

From the prometheus folder:

```
helm upgrade prometheus ^
prometheus-community/prometheus ^
-f values.yaml ^
-n prometheus
```

Then verify:

```
kubectl get pods -n prometheus
kubectl logs -n prometheus deployment/prometheus-server --tail=100
```

Expected result: Prometheus remains Running and reloads the generated configuration.

14. Lab 10 - Verify the Prometheus Target

```
kubectl port-forward -n prometheus svc/prometheus-server 9090:80
```

Open <http://localhost:9090> and inspect the Targets page. Look for:

```
job = postgres_exporter
state = UP
```

Test PromQL:

```
up{job="postgres_exporter"}
pg_up
```

Expected result: The target is UP and pg_up is 1.

15. Lab 11 - Add Patroni Monitoring

PostgreSQL exporter monitors PostgreSQL. Patroni itself exposes Prometheus-format metrics from its REST API, normally at port 8008 /metrics. Verify on each Patroni pod:

```
kubectl get pods -n postgres-ha -l application=spilo
```

```
kubectl exec -n postgres-ha POD_NAME -- curl -s http://localhost:8008/metrics
```

Create a headless metrics Service:

```
apiVersion: v1
kind: Service
metadata:
  name: patroni-metrics
  namespace: postgres-ha
spec:
  clusterIP: None
  selector:
    application: spilo
  ports:
    - name: metrics
      port: 8008
      targetPort: 8008
```

Save as patroni-metrics-service.yaml and apply:

```
kubectl apply -f patroni-metrics-service.yaml
kubectl get endpoints -n postgres-ha patroni-metrics
```

16. Lab 12 - Add Patroni Scrape Discovery to Prometheus values.yaml

Add a second job under the same extraScrapeConfigs block:

```
extraScrapeConfigs: |
- job_name: 'postgres_exporter'
  static_configs:
    - targets:
      - 'postgres-exporter-prometheus-postgres-exporter.postgres-ha.svc.cluster.local:80'

- job_name: 'patroni'
  kubernetes_sd_configs:
    - role: endpointslice
  namespaces:
```

```

names:
  - postgres-ha
relabel_configs:
  - source_labels:
      - __meta_kubernetes_service_name
    action: keep
    regex: patroni-metrics
  - source_labels:
      - __meta_kubernetes_endpointslice_port_name
    action: keep
    regex: metrics

```

Upgrade Prometheus again:

```

helm upgrade prometheus ^
prometheus-community/prometheus ^
-f values.yaml ^
-n prometheus

```

Expected result: Prometheus discovers the individual Patroni endpoints and scrapes /metrics.

17. Lab 13 - Grafana

If Grafana is already deployed, keep it. Otherwise install it:

```

helm repo add grafana https://grafana.github.io/helm-charts
helm repo update
kubectl create namespace monitoring
helm install grafana grafana/grafana -n monitoring

```

Expose Grafana to localhost with port-forward:

```

kubectl port-forward -n monitoring svc/grafana 3000:80

```

Open <http://localhost:3000>. Add Prometheus as a data source using the in-cluster URL:

```

http://prometheus-server.prometheus.svc.cluster.local

```

Expected result: Grafana Save & test succeeds.

18. Lab 14 - Grafana Dashboard Exercises

Panel	Query / purpose
PostgreSQL Exporter Up	<code>up{job="postgres_exporter"}</code>
PostgreSQL DB Up	<code>pg_up</code>
Patroni targets	<code>up{job="patroni"}</code>
Connections	Choose the <code>pg_stat_activity</code> metric exposed by your exporter build.
Transactions	Use <code>rate()</code> on <code>pg_stat_database</code> transaction counters exposed by the exporter.
Primary/Replica	Use the Patroni role metric actually returned by /metrics.
Replication	Use Patroni/PostgreSQL replication metrics available in the collected metric set.

Note: Metric names can vary by exporter/Patroni version. Confirm them in /metrics or Prometheus before building Grafana panels.

19. Lab 15 - Validate During Patroni Switchover

```

kubectl get pods -n postgres-ha -l application=spilo -L spilo-role

```

Perform the controlled Patroni switchover from the HA lab, then refresh Prometheus/Grafana. Patroni roles should change while monitoring targets remain available. The PostgreSQL exporter should continue to monitor the writable PostgreSQL service.

20. Troubleshooting

Problem	Check
Exporter pod not Running	kubectl logs -n postgres-ha deployment/postgres-exporter-prometheus-postgres-exporter
pg_up = 0	Datasource host, database, user/password, pg_hba/authentication and sslmode.
postgres_exporter target DOWN	Verify Service name/port and Prometheus target address.
host.docker.internal fails	Use Kubernetes Service DNS instead of routing through the host.
Prometheus Helm upgrade changes defaults	Review helm get values and use the existing release values as the base.
Patroni target absent	Check patroni-metrics EndpointSlices and port 8008.
Grafana No data	Run the PromQL directly in Prometheus first.

21. Final Validation Checklist

- ☐ Existing Prometheus installation remains Running.
- ☐ postgres-exporter was installed by Helm.
- ☐ Exporter datasource points to the correct writable PostgreSQL Service.
- ☐ Exporter /metrics is reachable on port 9187 through the appropriate Service mapping.
- ☐ Prometheus values.yaml contains the postgres_exporter scrape job.
- ☐ Prometheus Helm release upgrades successfully.
- ☐ Prometheus target postgres_exporter is UP.
- ☐ pg_up returns 1.
- ☐ Patroni /metrics is reachable from all three database pods.
- ☐ Prometheus discovers Patroni targets.
- ☐ Grafana can query Prometheus.
- ☐ Dashboard shows PostgreSQL and Patroni health.

22. Key Takeaway

The Prometheus values.yaml approach is valid. Keep the exporter and Prometheus chart values separate, prefer Kubernetes Service DNS for in-cluster scraping, and use extraScrapeConfigs (or the chart's scrapeConfigs map) to add monitoring jobs without discarding Prometheus's existing Kubernetes scrape configuration.